

MedAssistAI - Medical Symptom Analysis & Diseases Prediction System

Individual Contribution Report

Field	Details
Name	Koritala Lakshmi Sahitya
Role	Full Stack Developer
Evaluation Phase	Milestone - 4
Status	Completed

1. Overview

During the final milestone, the MedAssist AI application was finalized by integrating the developed frontend, backend, AI/ML services, APIs, and role-based healthcare workflows into a unified system. The primary focus was on deployment, functionality improvements, error resolution, cross-role integration, performance validation, and preparing the application for the final demonstration.

2. Deployment & Configuration

- Prepared the React/Vite frontend and Flask backend for deployment.
- Configured frontend-backend communication for the deployed environment.
- Replaced local development dependencies and localhost configurations where required.
- Configured environment variables for API keys and sensitive credentials.
- Ensured sensitive files such as `.env` were excluded from the shared/deployed project.
- Verified the application startup and deployment configuration.

3. Authentication & Demo Access

- Improved the authentication workflow for easier project demonstration.
- Implemented username/password-based access for demonstration accounts.

- Added role-specific demo accounts for Patient, Doctor, Lab Assistant, Receptionist, Pharmacy, Hospital Admin, and Super Admin.
- Verified correct dashboard redirection based on user roles.
- Resolved role authorization issues and incorrect unauthorized-page redirections.
- Preserved role-based access restrictions across the application.

4. AI/ML Integration

- Completed integration of the trained disease prediction model with the Flask backend.
- Connected the ML prediction API with the Patient Symptom Analysis interface.
- Implemented the complete prediction flow from symptom input to model prediction and frontend result display.
- Verified communication between the frontend, backend, and ML model.
- Added appropriate input validation and error handling for prediction requests.

5. Medical Report Analysis

- Improved the Medical Report Analysis workflow for uploaded healthcare documents.
- Added validation to distinguish medical reports from unrelated documents.
- Prevented irrelevant or non-medical files from generating inappropriate medical predictions.
- Improved processing and display of results for valid medical reports.
- Added error handling for invalid, unsupported, corrupted, or unreadable files.

6. AI Recommendation Improvements

- Identified and resolved the issue of similar diet recommendations being generated for different users.
- Improved the recommendation workflow to consider relevant user information.
- Maintained the existing BMI calculation functionality.
- Improved the personalization of health and diet recommendations.

7. AI Consultant & API Integration

- Integrated and tested the Grok API for the AI Consultant.
- Investigated repeated or identical chatbot responses for different questions.

- Verified the complete frontend → backend → Grok API → frontend communication flow.
- Improved request and response handling so that the current user query is processed correctly.
- Added appropriate error handling for API failures.
- Ensured API credentials are handled securely through backend environment configuration.

8. Nearby Hospitals

- Improved the Nearby Hospitals functionality using OpenStreetMap-based services.
- Used the user's live location for hospital discovery.
- Improved the hospital search logic to reduce irrelevant results.
- Configured the search to identify hospitals within an appropriate geographical radius.
- Preserved the existing live-location detection functionality.

9. Role-Based & Cross-Role Integration

- Verified workflows across Patient, Doctor, Lab Assistant, Receptionist, Pharmacy, Admin, and Super Admin portals.
- Improved synchronization of relevant patient information between authorized roles.
- Verified that laboratory reports can be accessed by relevant doctors.
- Verified that appointments are reflected in the appropriate patient and doctor workflows.
- Ensured prescription and medication information can flow between relevant healthcare roles.
- Tested Super Admin features and role permissions.

10. Application Optimization

- Removed the unnecessary **Image Prediction/Analysis** feature.
- Removed related unused routes, components, and dependencies where applicable.
- Removed or improved non-functional placeholder features.
- Improved navigation and overall usability.
- Preserved all required healthcare workflows and existing functionality.

11. Testing & Error Resolution

Performed final functional testing of major application workflows, including:

- Authentication and demo login
- Role-based dashboards
- Disease prediction
- Medical report analysis
- AI recommendations
- AI Consultant
- Appointment booking
- Laboratory report upload
- Pharmacy workflow
- Nearby Hospitals
- Admin functions
- Super Admin functions
- Cross-role data synchronization
- Frontend-backend communication

Identified and resolved integration, API, authorization, and functionality-related issues while ensuring that existing working features were not affected.

12. Final Outcomes

- Developed a unified and deployment-ready full-stack healthcare application.
- Successfully integrated the trained ML disease prediction model with the web application.
- Completed frontend, backend, database, API, and role-based workflow integration.
- Improved reliability of AI-powered features and external API services.
- Implemented accessible demonstration login through predefined role-based accounts.
- Improved cross-role data synchronization and healthcare workflows.
- Enhanced application usability by removing unnecessary functionality and resolving non-functional features.
- Conducted final testing and validation across the major application modules.
- Prepared the MedAssist AI system for final demonstration and project submission.

13. Conclusion

The final milestone completed the development and integration of the **MedAssist AI: Medical Symptom Analysis & Disease Prediction System**. The frontend, Flask backend, database, AI/ML models, external APIs, authentication, and role-based healthcare workflows were brought together into a unified application. Through final testing, debugging, deployment preparation, and feature optimization, the system was made more reliable, scalable, and suitable for real-world demonstration. The completed application provides an integrated platform for symptom analysis, disease prediction, medical report analysis, healthcare recommendations, appointments, and coordinated interaction between different healthcare roles.